sqlmap integration and operator guide

Date: 2026-10-27

Scope: Cerberus pentest framework; sqlmap is a separately installed external tool.

> Only test systems for which you have explicit written authorization. Even SQL injection detection can change data, interrupt service, or trigger security controls. The restrictions below reduce risk; they do **not** guarantee that a target will remain unchanged.

1. What was added

- `backend/app/ai_engine/pentest/tools/sqlmap_adapter.py`: external CLI adapter, private per-scan request/output files, bounded process lifecycle, and final-summary parser.
- `backend/app/ai_engine/pentest/modules/tools/sqlmap_scanner.py`: scope and impact checks, scanner settings, normalized SQL-injection findings, and shared-state output.
- `backend/tests/unit/test_sqlmap_integration.py`: adapter, safety, error handling, cleanup, concurrency, coexistence, and database-persistence tests.
- `scripts/live_tests/sqlmap_framework_smoke.py`: disposable local SQLite lab for real sqlmap, optional real Nmap, and a **simulated** Metasploit XML-RPC service.
- `docs/SQLMAP_INTEGRATION.md`: this operator and verification guide.
- `backend/app/core/config.py` and `backend/.env.example`: opt-in sqlmap settings.
- `backend/app/ai_engine/pentest/tools/__init__.py` and `backend/app/ai_engine/pentest/__init__.py`: public adapter exports.
- `backend/app/ai_engine/pentest/modules/tools/__init__.py` and `scripts/live_tests/pentest_framework_smoke.py`: updated tool references and discovery check.

The existing Nmap and Metasploit adapter/scanner implementation files were not modified. The registry discovers three adapters (`nmap`, `sqlmap`, `metasploit`) and nine attack modules, including `sqlmap_scanner`. API endpoint and AI-agent wiring remain a separate phase.

2. Supported behavior and limits

Capability	Current integration
Target	One scope-approved HTTP(S) URL per scanner invocation; exact hostname and effective port are checked.
Request	GET query parameters, or URL-encoded form POST. POST requires at least `limited` impact in the RoE.
Parameters	All supported request parameter names, or the selected `sqlmap_parameters` list.
Techniques	Boolean-based and error-based (`B`, `E`, or `BE`). Risk is fixed at 1.
Test level	1 by default. Levels 2–5 require at least `limited` impact.
Authentication	Approved `auth_headers`, `auth_token`, or `auth_cookies` in `PentestContext`. No credentials are placed in the subprocess arguments.
Rate/time	One sqlmap HTTP thread; delay is at least the framework/approved-scope delay; configurable scan and outer-engine timeouts.
Redirects	HTTP redirects and refresh redirections are not followed by the tested sqlmap versions.
Output	SQLMap's final summary becomes a HIGH SQL Injection finding with CWE-89 and hashed observation evidence; a negative scan is INFO, and an incomplete scan is ERROR/inconclusive. Findings should be independently reviewed.
Isolation	Each scan uses its own temporary request file and output directory; the process is terminated on timeout/cancellation, and temporary files are removed on normal cleanup.

Not exposed: Arbitrary sqlmap flags, crawling, JSON request bodies, database dumping/enumeration, SQL/OS shells, file access, stacked queries, time-based and UNION techniques. This is not an implementation of every sqlmap feature.

A finding's `target` and existing framework audit events can contain the target URL and its query values. Avoid putting secrets in URLs, and protect logs/results. The new shared-state records omit query values, auth data, response bodies, and raw SQL payloads. Temporary-file cleanup is best effort; a hard machine crash can leave OS temporary files behind.

3. Install and configure

First complete the project's normal backend setup, including `backend/.venv`, dependencies, and a real `SECRET_KEY` of at least 32 characters in `backend/.env`. Do not commit `.env`. From a VS Code terminal opened at the **Cerberus project root**:

MacBook / zsh

```
brew install sqlmap
sqlmap --version
# Optional for the three-tool smoke test, if not already installed:
brew install nmap
nmap --version
```

Windows / PowerShell

If the official repository has not already been cloned:

```
git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git "$HOME\sqlmap"
.\backend\.venv\Scripts\python.exe "$HOME\sqlmap\sqlmap.py" --version
$env:SQLMAP_PATH = "$HOME\sqlmap\sqlmap.py"
```

The environment variable above applies to the current terminal. Alternatively, set an absolute `SQLMAP_PATH` in `backend/.env`; use your actual path, not a placeholder.

In `backend/.env`, add or update:

```
SQLMAP_ENABLED=true
SQLMAP_PATH=sqlmap
SQLMAP_TIMEOUT=90
```

For a clone, set `SQLMAP_PATH` to the absolute path of its `sqlmap.py`; the adapter runs it with the same Python interpreter as the backend. Restart the backend process after changing `.env`. `SQLMAP_ENABLED` defaults to false. Missing/disabled sqlmap yields an INFO finding, not a crash. The smoke script temporarily enables sqlmap **in its own process** and does not edit `.env`.

The framework's outer `PENTEST_MODULE_TIMEOUT` defaults to 120 seconds and includes the version check and scanner run. If an approved test legitimately needs a longer budget, configure both `SQLMAP_TIMEOUT` and `PENTEST_MODULE_TIMEOUT` accordingly before starting the backend. An outer timeout can end a scan before its sqlmap-specific timeout expires.

4. Verify locally

Real sqlmap and Nmap on a disposable local lab

MacBook / zsh, from the project root:

```
./backend/.venv/bin/python scripts/live_tests/sqlmap_framework_smoke.py
```

Windows / PowerShell, from the project root, after setting `$env:SQLMAP_PATH` as above:

```
.\backend\.venv\Scripts\python.exe .\scripts\live_tests\sqlmap_framework_smoke.py
```

The script binds only to `127.0.0.1`, uses temporary SQLite databases, tests vulnerable GET and POST requests and a parameterized control, checks off-scope redirect handling, and runs two independent sqlmap engagements concurrently. When Nmap is installed, it also schedules real Nmap and real sqlmap with a **fake** Metasploit service in the same plan. If Nmap is missing, it prints `SKIP` for Nmap. Expected output includes:

```
PASS: ... real sqlmap detected GET SQLi
NOTE: Nmap is real when present; the Metasploit RPC service is a test double.
PASS: parameterized GET control produced no actionable finding
PASS: real sqlmap detected SQLi in an authorized form POST
PASS: out-of-scope HTTP redirects and HTML refreshes were not followed
PASS: two real sqlmap scans ran concurrently in isolated engagements
PASS: real Nmap + real sqlmap + fake Metasploit ran in separate concurrent engagements
```

No real Metasploit connectivity is established by this script.

Framework and tool unit tests without a database

MacBook / zsh:

```
cd backend
./.venv/bin/python -m pytest --noconftest -q -k 'not persist and (sqlmap or pentest)' tests/unit/
cd ..
```

Windows / PowerShell:

```
Set-Location backend
.\.venv\Scripts\python.exe -m pytest --noconftest -q -k "not persist and (sqlmap or pentest)" tests/unit/
Set-Location ..
```

The `not persist` filter excludes database persistence tests. The full backend suite was also run here against a **dedicated PostgreSQL test database**: 250 passed, with 52 existing warnings. The test fixture creates and drops tables. Before running the full suite yourself, inspect `backend/tests/conftest.py` and ensure its derived database URL targets a disposable test database, never production.

5. Use from the framework

Run the following **Python code** from an application or script configured with the backend imports and settings. Replace `authorized.example` and the path with a target explicitly approved for your engagement; the example hostname is not a live test target.

```
import asyncio
from app.ai_engine.pentest import (
    ExecutionEngine, ExecutionScheduler, PentestContext,
    ScheduledModule, ScopeConfig,
)

async def run_approved_plan():
    context = PentestContext(
        target="https://authorized.example/search?id=1",
        scope=ScopeConfig(
            allowed_targets=["authorized.example"],
            allowed_ports=[443],
            max_impact="proof_of_concept",
            request_delay=0.25,
        ),
        config={
            "nmap_ports": [443],
            "sqlmap_parameters": ["id"],
            "sqlmap_techniques": "BE",
            "sqlmap_level": 1,
            "sqlmap_timeout": 90,
        },
    )
    report = await ExecutionScheduler(ExecutionEngine()).run(
        [ScheduledModule("nmap_scanner"), ScheduledModule("sqlmap_scanner")],
        context,
    )
    for finding in report.findings:
        print(finding.severity.value, finding.title)
    print("sqlmap results:", context.get_shared("sqlmap_results", []))

asyncio.run(run_approved_plan())
```

Enable and install each tool separately. The scheduler is intentionally **sequential within one engagement plan**; it does not run those modules simultaneously. Independent engagements can run concurrently, and the new sqlmap adapter isolates their temporary files. Do not assume there is a global request-rate limit across independent engagements. Check each finding's severity, not just `report.errors`, to identify module failures.

For form POST, use `max_impact="limited"` or higher and set:

```
context.config.update({
    "sqlmap_method": "POST",
    "sqlmap_data": "id=1",  # URL-encoded form; replace with authorized test data
    "sqlmap_parameters": ["id"],
})
```

Approved authentication can be supplied through `context.auth_headers`, `context.auth_token` (Bearer), or `context.auth_cookies`. Do not put real tokens in source code. Host/routing, content-length, method-override, and similar headers are refused. The adapter does not consume Nmap's shared ports automatically; select an explicitly approved web URL/port.

`context.get_shared("sqlmap_results", [])` contains one redacted record per completed scan. Actionable SQLMap findings use the existing `finding_to_vulnerability_dict` and `persist_findings` bridge; persistence requires a valid engagement and database session. That bridge is not the same as API/AI-agent wiring, which remains future work.

6. Existing limitations requiring separate attention

1. **Real Metasploit is not verified.** The existing adapter uses XML-RPC; the real Metasploit Framework RPC service uses MessagePack. Tests in this integration use a fake XML-RPC service. The earlier claim that real Metasploit RPC had been verified was wrong.
2. **Existing Metasploit scope bypass:** `msf_options` can override the target host and port options. A controlled test against the fake service confirmed this. Do not rely on that module's scope enforcement until repaired.
3. **Existing Nmap NSE gate:** wildcard script selections such as `*` pass the current keyword-based check and can include intrusive/DoS NSE scripts. Do not use wildcard NSE selection. The earlier blanket claim that all NSE safety was enforced was too broad.
4. The existing common scope matcher uses substring matching. The new sqlmap module adds stricter exact-host and effective-port checks, but does not change scope behavior in other modules.

These pre-existing issues were not changed as part of the sqlmap work. The combined stack should not be treated as production-ready until they are fixed and real Metasploit is tested.

7. Verification record

On 2026-10-27, the complete backend test suite reported **250 passed** with PostgreSQL; Ruff passed on the affected Python files; mypy passed on the pentest package; Black passed on the sqlmap code, tests, smoke scripts, and package exports. The pre-existing formatting of `app/core/config.py` was left unchanged. The disposable lab passed with real Nmap 7.95 and real sqlmap 1.9.6 stable, and again with the sqlmap upstream development snapshot `f8a42cf016d22ad4ce1c83ade21fafb18e010835`. Metasploit in those joint runs was a fake service, not the real Framework. Verification was performed in a Linux-based sandbox; the macOS and Windows installation commands were not executed on those operating systems.

Sources: sqlmap repository, sqlmap usage, Rapid7 RPC API.